

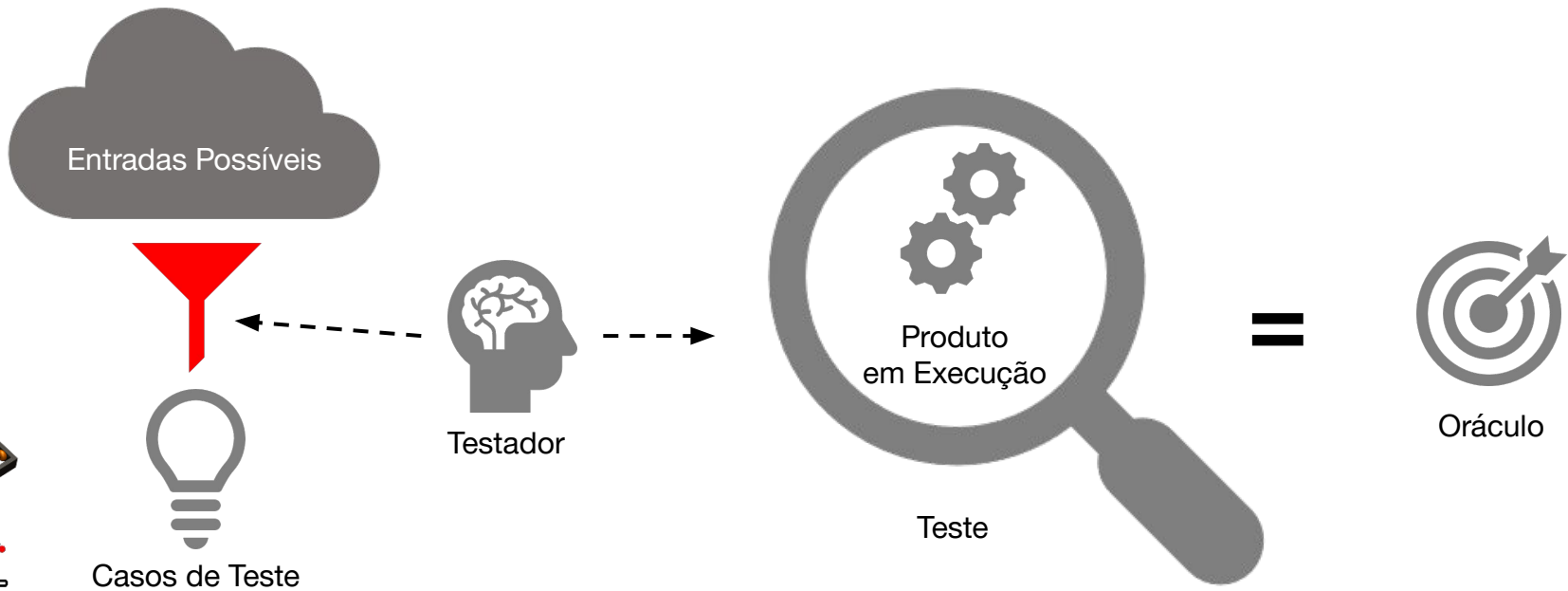
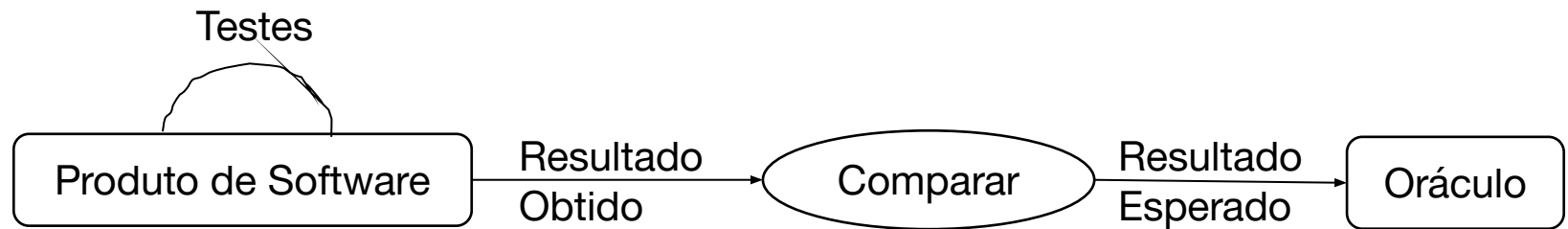
# Testes Automatizados

Prof. Breno de França

Prof. Bruno Cafeo

`cafeo@unicamp.br`

# *O que é teste de software?*



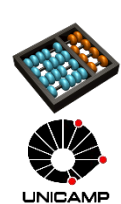
# Testes Automatizados

Ajudam a manter uma noção sobre o **estado funcional** da base de código

Identificam módulos com **baixa testabilidade** e oportunidades de **refatoração**

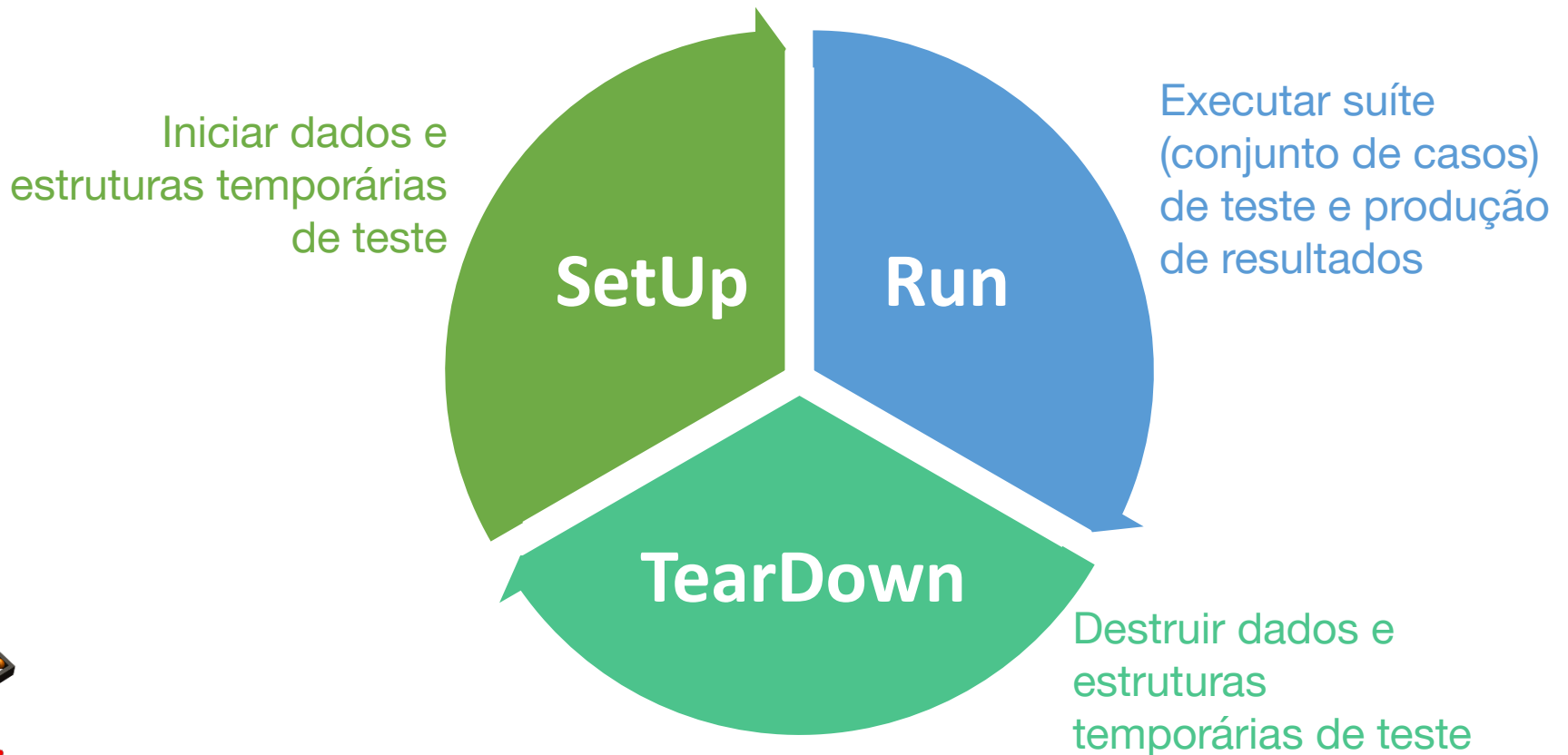
Aceleram testes de **regressão**

Permitem **testes contínuos**



# Testes Automatizados

## Funcionamento:



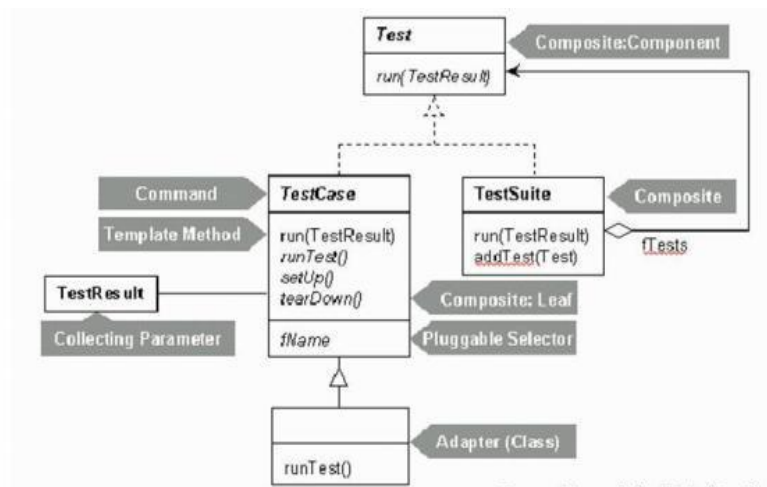
# Testes Automatizados

Exemplo simples:

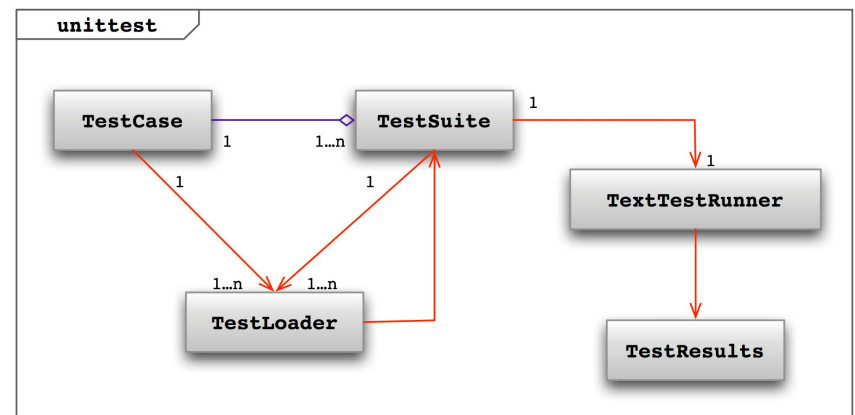


# Testes Automatizados

Ferramentas de Testes Unitários, em geral, incluem:

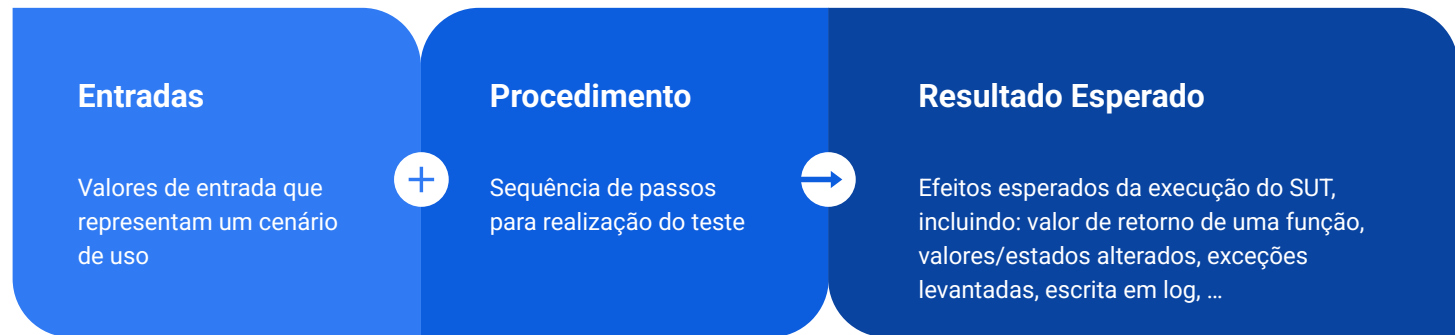


Fonte: Manual do JUnit (Cooks Tour)



# Testes Automatizados

Um caso de teste inclui:

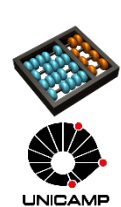


# Asserções

Um módulo de teste pode possuir diversos (casos de) testes, organizados em métodos

Métodos recebem entrada (de teste), possuem alguma lógica para inicializar o teste (*set up*), executam a unidade em questão, e o **resultado (saída/retorno) esperado é analisado**

Essa análise é feita por meio de **asserções**





# Assertões

Assumem alguma **verdade** sobre o estado da unidade

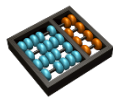
## Exemplos:

AssertTrue: `sut.method(param1, param2) = 1` → Se **não** for 1 o teste falha

AssertFalse: `sut.method(param1, param2) = 1` → Se for 1 o teste falha

AssertException: espera que alguma exceção ocorra

....

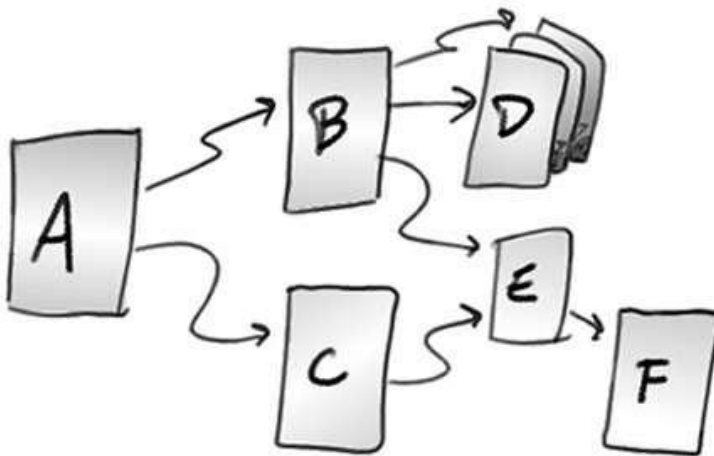


# Assertões

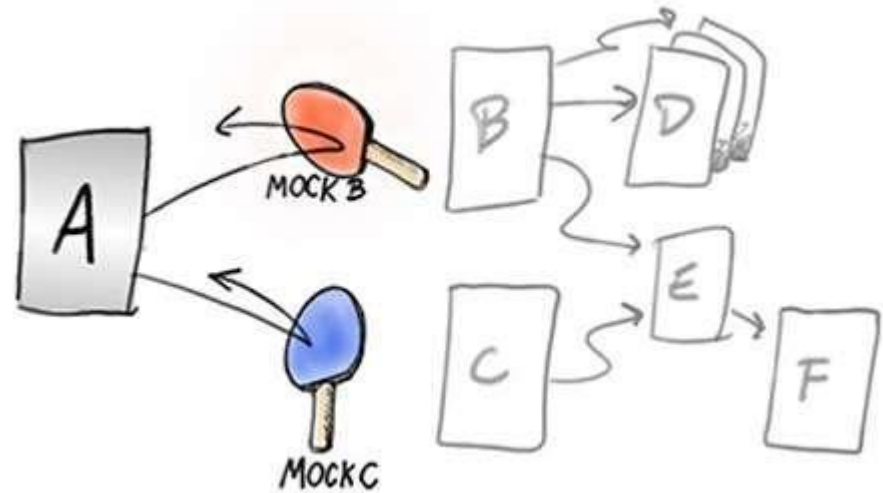
## Exemplos (Python):

| Method                                                 | Checks that                          |
|--------------------------------------------------------|--------------------------------------|
| <a href="#"><code>assertEqual(a, b)</code></a>         | <code>a == b</code>                  |
| <a href="#"><code>assertNotEqual(a, b)</code></a>      | <code>a != b</code>                  |
| <a href="#"><code>assertTrue(x)</code></a>             | <code>bool(x)</code> is True         |
| <a href="#"><code>assertFalse(x)</code></a>            | <code>bool(x)</code> is False        |
| <a href="#"><code>assertIs(a, b)</code></a>            | <code>a</code> is <code>b</code>     |
| <a href="#"><code>assertIsNot(a, b)</code></a>         | <code>a</code> is not <code>b</code> |
| <a href="#"><code>assertIsNone(x)</code></a>           | <code>x</code> is None               |
| <a href="#"><code>assertIsNotNone(x)</code></a>        | <code>x</code> is not None           |
| <a href="#"><code>assertIn(a, b)</code></a>            | <code>a</code> in <code>b</code>     |
| <a href="#"><code>assertNotIn(a, b)</code></a>         | <code>a</code> not in <code>b</code> |
| <a href="#"><code>assertIsInstance(a, b)</code></a>    | <code>isinstance(a, b)</code>        |
| <a href="#"><code>assertNotIsInstance(a, b)</code></a> | <code>not isinstance(a, b)</code>    |

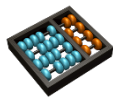
# Dublês de Teste (*Test Doubles*)



Sistema Real

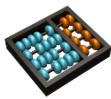
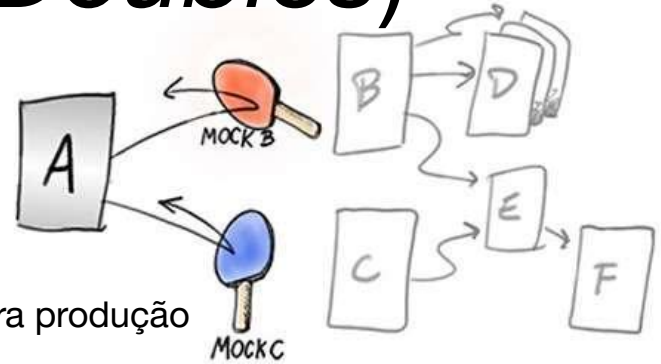


Teste com dublês



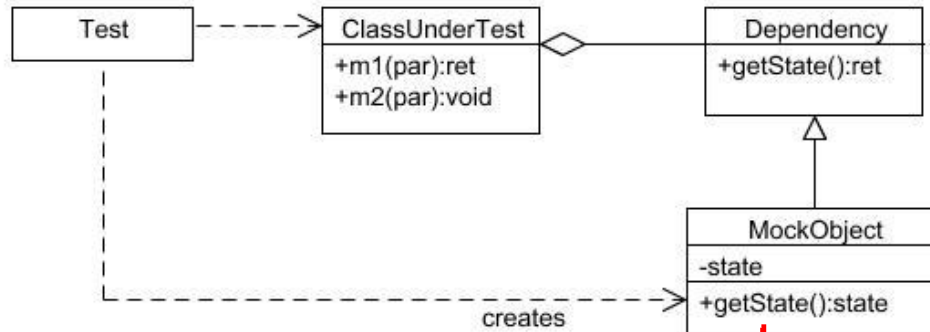
# Dublês de Teste (*Test Doubles*)

- *Dummy*
  - Passados mas nunca utilizados.
  - Utilizados apenas para preencher parâmetros
- Objetos Falsos
  - Possuem implementação, mas com atalhos inviáveis para produção
  - Ex.: banco de dados em memória
- *Stubs*
  - Provêem respostas pré-prontas
  - Retorno é programado para retornar uma resposta em particular
  - Verifica estado do módulo em teste
- Espiões
  - *Stubs* que guardam informação dependendo da forma que são chamados
  - Ex.: serviço de e-mail que conta quantas mensagens enviou
- *Mocks*
  - Objetos pré-programados com expectativas que formam uma especificação das chamadas que deveriam receber
  - Verifica comportamento do módulo em teste
  - Comuns no contexto de TDD/BDD

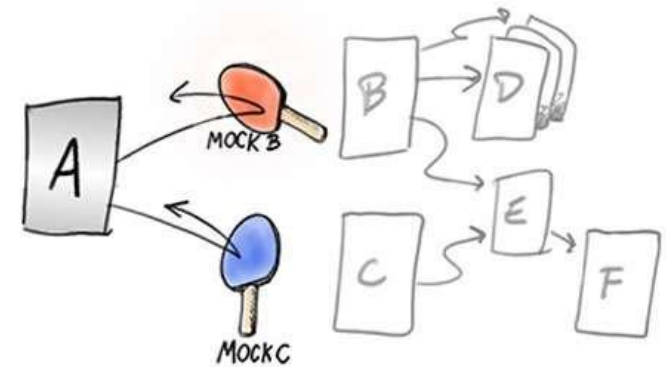
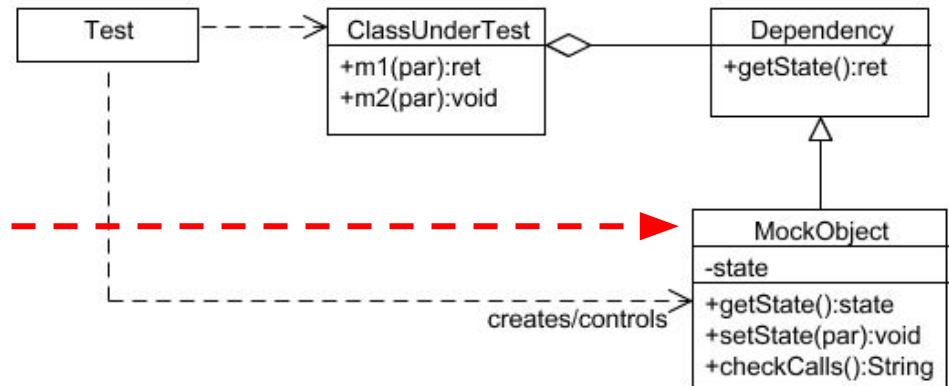


# Dublês de Teste (*Test Doubles*)

*Implementados por meio de  
Realização de Interfaces ou Herança*



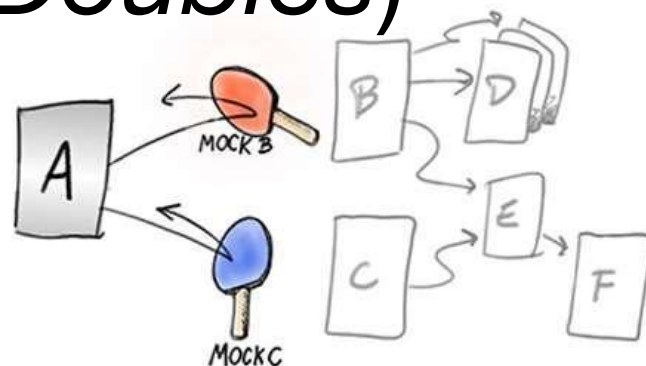
Flexível, permite outras  
situações de teste



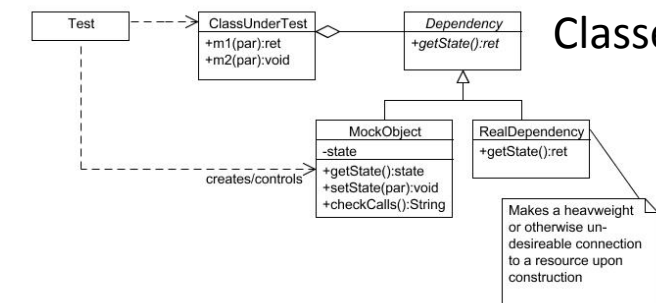
Fonte:  
<http://www.netobjectives.com/PatternRepository/index.php?title=TheMockObjectPattern>

# Dublês de Teste (*Test Doubles*)

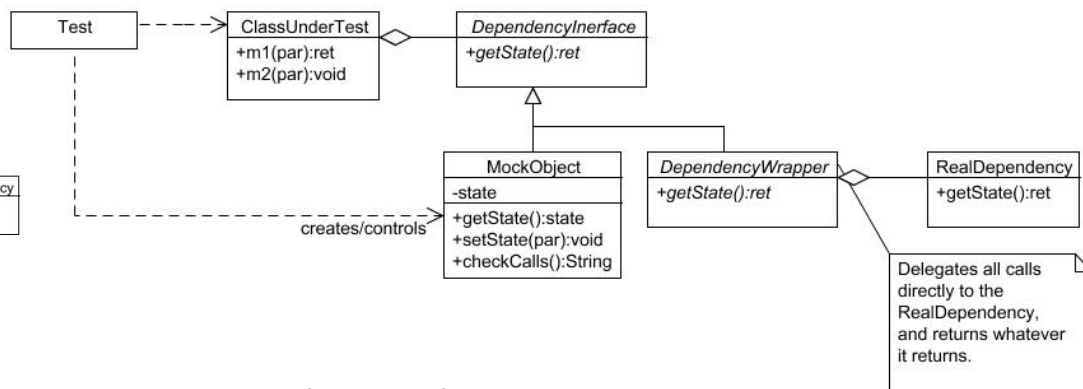
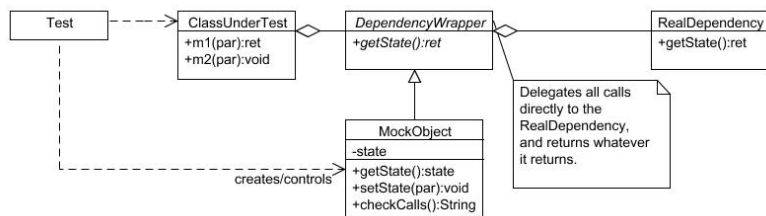
*Tornando o código mais testável*



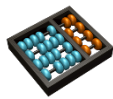
## Classe Abstrata



## Wrapper



## Classe Abstrata + Wrapper



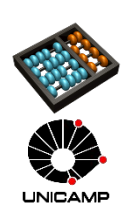
Fonte:  
<http://www.netobjectives.com/PatternRepository/index.php?title=TheMockObjectPattern>

# Exemplo

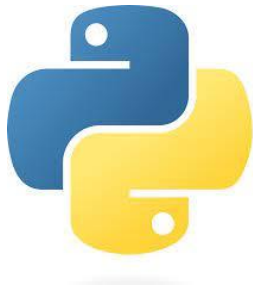
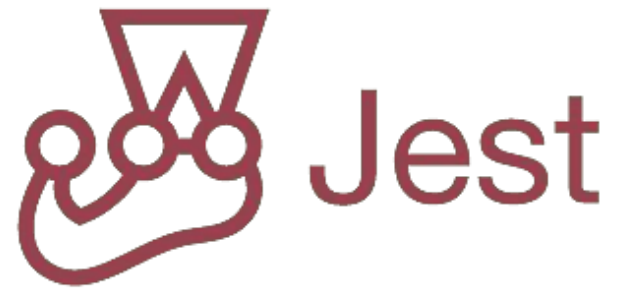
```
@Test(expected = IllegalStateException.class)
public void
givenMethodIsConfiguredToThrowException_whenCallingMetho
d_thenExceptionIsThrown() {

    MyList listMock = Mockito.mock(MyList.class);
    when(listMock.add(anyString()))
        .thenThrow(IllegalStateException.class);

    assertThrows(IllegalStateException.class,
        () -> listMock.add(randomAlphabetic(6))
    );
}
```



# Ferramentas



Python Mock

